

LJ INSTITUTE OF COMPUTER APPLICATIONS

LJ University, Ahmedabad
Department of Computer Applications

M1 DESIGN DOCUMENT

BazzaBuddy AI

Smart Kirana Store Assistant – Owner/Admin Console

Subject Code	UDP-501
Programme	B.Sc. Information Technology – Semester V
Subject	User Defined Project – I (UDP-I) Build Lab I: React.js + Node.js
Project Title	BazzaBuddy AI – Smart Kirana Store Assistant (Owner/Admin Console)
Milestone	M1 – Design Document (Due: Session S10)
Marks	10 Marks (IPE – Internal Project Evaluation)
Academic Year	2025–26 Odd Semester
Date of Submission	-----
Group Members	1. Riddhi Makwana 2. Bhumi Chauhan
GitHub Repository	https://github.com/riddhi1120/BazzarBuddy-AI.git
GitHub Tag	v1.0-design-doc

Document Version: 1.0 | Format: PDF + GitHub Tagged Release

Table of Contents

Table of Contents	2
1. Project Overview.....	4
1.1 Problem Statement.....	4
1.2 Proposed Solution	4
1.3 Technology Stack.....	4
2. Software Requirements Specification (SRS)	5
2.1 Scope.....	5
2.2 Definitions & Abbreviations	5
2.3 Functional Requirements (FR).....	5
2.4 Non-Functional Requirements (NFR)	6
2.5 Constraints.....	7
3. Use Case Diagram	8
3.1 Actors.....	8
3.2 Use Cases	8
3.3 Key Relationships.....	9
4. Data Flow Diagrams (DFD)	10
4.1 DFD Level 0 – Context Diagram	10
4.2 DFD Level 1 – Process Decomposition.....	11
4.3 Data Stores	11
5. Entity-Relationship (ER) Diagram.....	13
5.1 Entities and Attributes.....	13
5.2 Relationships and Cardinality	13
6. Database Schema	15
6.1 User Schema	15
6.2 Product Schema	15
6.3 Batch Schema	15
6.4 Sale & SaleItem Schema	16
6.5 Customer & CreditTransaction Schema.....	16
6.6 Supplier & Category Schemas	16
6.7 Normalisation Notes	16
7. System Architecture Diagram.....	18
7.1 Architecture Components.....	18
7.2 API Routes.....	18
8. Wireframes	21
8.1 Screen 1 – Login (Route: /login)	21
8.2 Screen 2 – Owner Dashboard (Route: /dashboard)	21
8.3 Screen 3 – Inventory / Product List (Route: /inventory).....	22
8.4 Screen 4 – Billing / POS (Route: /billing).....	23
8.5 Screen 5 – Customer Credit (Udhar) Ledger (Route: /customers).....	23

8.6 Screen 6 – Reports & AI Suggestions (Route: /reports).....	24
9. Project Gantt Chart.....	26
9.1 Milestones.....	26
10. M1 Design Document Submission Checklist	27

1. Project Overview

1.1 Problem Statement

Most kirana (neighbourhood grocery) store owners still run their shop using paper notebooks or basic mobile apps with no structure. This causes several recurring problems:

- Products frequently go out of stock because the owner has no visibility into remaining quantity until the shelf is already empty.
- Perishable and dated products (packaged food, medicines, dairy) expire unnoticed, causing direct financial loss.
- Customer credit (udhar) is tracked in a notebook, making it easy to lose track of who owes how much, and hard to follow up.
- There is no sales analysis — the owner cannot tell which products are profitable, which are dead stock, or how sales trend over time.
- Restocking decisions are guesswork; owners either over-order (cash locked in unsold stock) or under-order (lost sales).

1.2 Proposed Solution

BazzaBuddy AI is a web-based admin console — built for the shop owner (and optionally shop staff) only — that digitises day-to-day store operations and adds an AI layer on top of the owner's own sales data. It is NOT a customer-facing ordering or delivery app (unlike Blinkit/Zepto); customers never log in or place orders through it. The system covers billing, inventory with batch/expiry tracking, customer credit (udhar) ledgers, sales analytics, and AI-driven suggestions for restocking, expiry risk, and best-sellers — all from a single dashboard the owner checks each morning.

1.3 Technology Stack

Layer	Technology	Purpose
Frontend	React.js + Vite + Tailwind CSS	UI components, routing, API calls
State / Routing	useState, useEffect, React Router v6	State management & SPA routing
HTTP Client	Axios	REST API calls from React to Express
Backend	Node.js + Express.js	REST API server, business logic
Database	MongoDB Atlas + Mongoose	Persistent data storage (ODM)
AI / Analytics Engine	Node.js statistical logic (+ optional Python/scikit-learn microservice)	Reorder prediction, expiry-risk scoring, best-seller detection
Authentication	JWT + bcrypt	Secure login, role-based access (Owner / Staff)
Deployment	Vercel (Frontend) + Render (Backend)	Live production hosting
Version Control	Git + GitHub	Source control, collaboration

2. Software Requirements Specification (SRS)

2.1 Scope

BazzaBuddy AI is a web-based admin console for a single kirana store, used by the Owner and optionally by Staff/Cashiers, on a computer or tablet at the billing counter. The system covers billing (POS-style), inventory with supplier and batch/expiry tracking, customer credit (udhar) management, sales analytics, and AI-based reorder/expiry/best-seller suggestions. The system does NOT cover: a customer-facing app or website, online ordering, delivery/logistics, online payment gateway processing (payment mode is recorded, not processed), or multi-store/franchise management in this version.

2.2 Definitions & Abbreviations

Term / Abbreviation	Definition
Owner	The store owner; has full access to every module.
Staff / Cashier	Shop staff with billing-only access; cannot edit inventory, credit limits, or view reports.
Udhar	Hindi/Gujarati term for customer credit — goods sold now, paid for later.
Batch	A specific lot of stock received from a supplier on a given date, with its own expiry date.
SKU / Barcode	Stock Keeping Unit — unique code identifying a product for scanning.
POS	Point of Sale — the billing screen where a sale is recorded.
Reorder Level	The minimum quantity of a product below which a low-stock alert is triggered.
AI Engine	The module that analyses sales history to suggest reorder quantities, expiry-risk items, and best-sellers.
JWT	JSON Web Token — used for stateless authentication.
ODM	Object Document Mapper — Mongoose library maps MongoDB documents to JS objects.

2.3 Functional Requirements (FR)

Each requirement uses the format: FR-ID | Actor | The system SHALL...

FR-ID	Actor	Requirement (System SHALL...)	Priority
FR-01	Owner	Allow the Owner to register/log in with name, email, password; issue a role-tagged JWT (Owner or Staff).	HIGH
FR-02	Owner	Allow the Owner to create Staff/Cashier accounts and set their access to billing-only.	HIGH
FR-03	Owner/Staff	Authenticate users via email and password; return a JWT token valid for 24 hours.	HIGH
FR-04	Owner	Allow the Owner to add/edit/delete a product with name, barcode, category, cost price, selling price, unit, and reorder level.	HIGH
FR-05	Owner	Allow the Owner to record incoming stock as a batch: supplier, quantity, purchase price, and expiry date.	HIGH

FR-ID	Actor	Requirement (System SHALL...)	Priority
FR-06	Owner/Staff	Allow searching or barcode-scanning a product on the billing screen with results in under 1 second.	HIGH
FR-07	Owner/Staff	Allow generating a bill with multiple line items, auto-calculated total, and a selected payment mode (Cash / UPI / Card / Credit).	HIGH
FR-08	System	On bill completion, automatically deduct sold quantity from the product's batch(es), oldest expiry first (FIFO).	HIGH
FR-09	System	Automatically raise a low-stock alert when a product's quantity falls below its reorder level.	HIGH
FR-10	System	Automatically raise an expiry alert for any batch expiring within the next 7 days.	HIGH
FR-11	Owner	Allow the Owner to add a customer profile with name, phone number, and an optional credit limit.	HIGH
FR-12	Owner/Staff	Allow recording a credit (udhar) sale against a customer, blocked if it would exceed the customer's credit limit.	HIGH
FR-13	Owner	Allow the Owner to record a full or partial repayment against a customer's outstanding credit and update the ledger balance.	HIGH
FR-14	Owner	Display a customer-wise credit ledger showing all credit and repayment transactions with running balance.	MEDIUM
FR-15	System	AI Engine SHALL analyse the last 30 days of sales per product and suggest a reorder quantity.	MEDIUM
FR-16	System	AI Engine SHALL flag slow-moving batches nearing expiry so the Owner can discount or prioritise their sale.	MEDIUM
FR-17	System	AI Engine SHALL identify best-selling and dead-stock products from sales history.	MEDIUM
FR-18	Owner	Display an Owner Dashboard with today's sales, profit, low-stock count, expiry alerts, pending credit, best-sellers, and AI suggestions.	HIGH
FR-19	Owner	Allow the Owner to generate and export a sales/profit report (daily / weekly / monthly) as PDF or CSV.	MEDIUM

2.4 Non-Functional Requirements (NFR)

NFR-ID	Category	Requirement	Metric / Target
NFR-01	Performance	Product search/barcode lookup on the billing screen must respond quickly, even for a catalogue of up to 5,000 products.	Response < 1s at p95
NFR-02	Security	All user passwords must be hashed using bcrypt with a minimum salt round of 10 before storage.	bcrypt, salt = 10
NFR-03	Security	All protected API routes must validate the JWT and enforce role (Owner vs Staff) before processing.	401/403 if invalid/unauthorised
NFR-04	Availability	System should support at least 5 concurrent devices/terminals at the counter without performance degradation.	5 concurrent terminals
NFR-05	Usability	Dashboard, alerts, and AI suggestions must load within 2 seconds of login.	Load < 2s

NFR-ID	Category	Requirement	Metric / Target
NFR-06	Maintainability	Codebase must pass ESLint with zero errors before deployment.	0 ESLint errors
NFR-07	Scalability	MongoDB schema must support future addition of multiple store branches without schema migration.	Extensible schema (storeId field ready)

2.5 Constraints

- Admin/Owner-side only — no customer-facing ordering, cart, or delivery app is built (explicitly out of scope, unlike Blinkit/Zepto).
- Maximum 3 students per group; individual contribution tracked via GitHub commits.
- Technology stack is fixed: React.js (Vite), Node.js + Express.js, MongoDB Atlas.
- Online payment gateway integration is OUT OF SCOPE for this milestone — payment mode is recorded, not processed online.
- Mobile native app is OUT OF SCOPE; responsive web design only.
- All design diagrams must be submitted in /docs/diagrams/ on GitHub before S10.

3. Use Case Diagram

The Use Case diagram below identifies the system actors and the operations they can perform within BazzaBuddy AI. The actual diagram must be drawn in draw.io and uploaded to /docs/diagrams/use-case.png on GitHub.

3.1 Actors

Actor	Type	Description
Owner	Primary	Store owner. Full access — inventory, billing, credit, reports, AI suggestions, staff accounts.
Staff / Cashier	Primary	Shop staff. Billing-only access — search products, generate bills, record credit sales.
System (Scheduled)	Secondary	Automated background process that checks stock levels, flags expiring batches, and runs the AI engine daily.

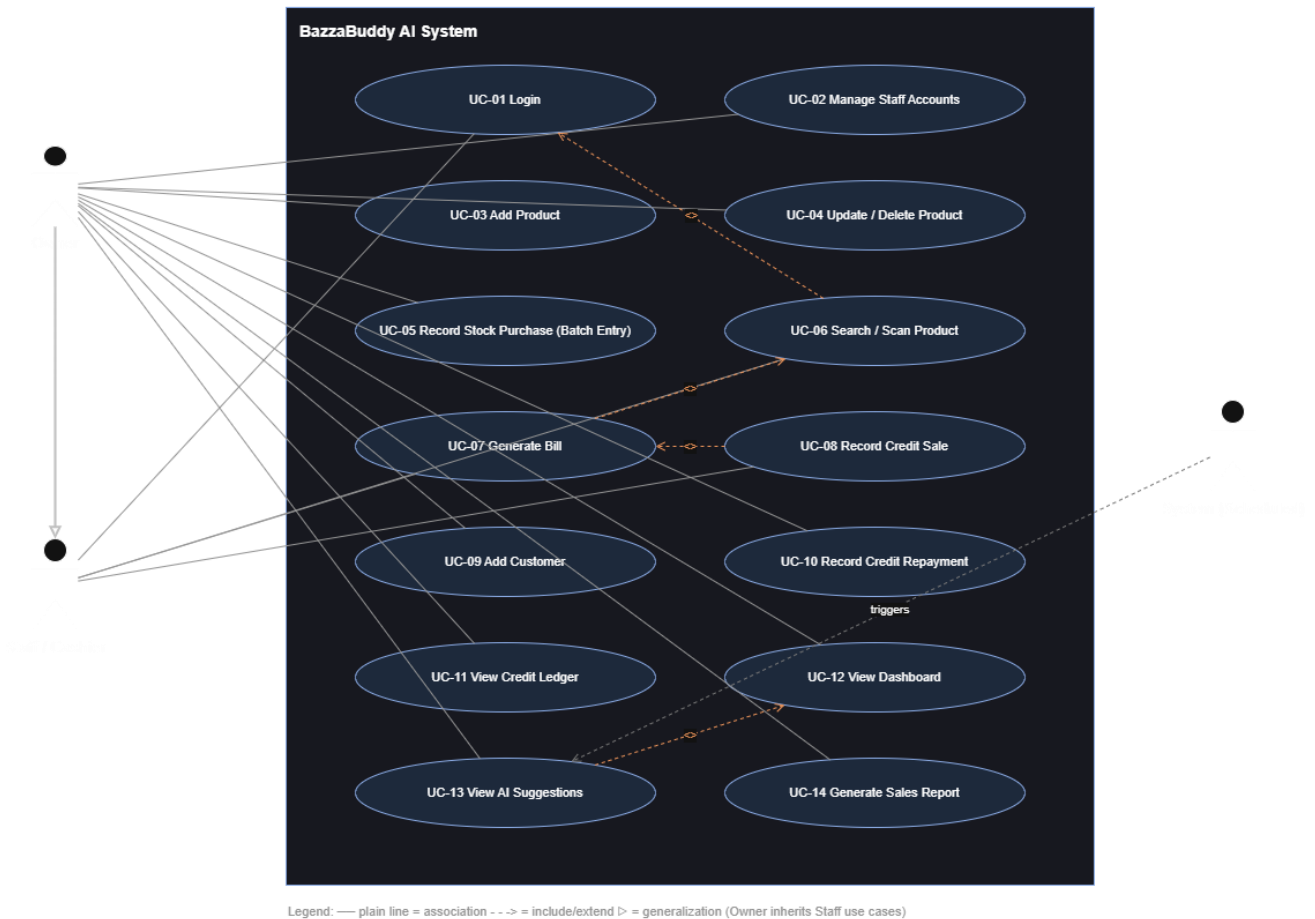
3.2 Use Cases

UC-ID	Use Case Name	Primary Actor	Brief Description
UC-01	Login	Owner / Staff	User authenticates with email and password; receives a role-tagged JWT.
UC-02	Manage Staff Accounts	Owner	Owner creates or deactivates Staff/Cashier accounts.
UC-03	Add Product	Owner	Owner adds a new product to the catalogue.
UC-04	Update / Delete Product	Owner	Owner edits or removes a product record.
UC-05	Record Stock Purchase (Batch Entry)	Owner	Owner records a new batch received from a supplier with expiry date.
UC-06	Search / Scan Product	Owner / Staff	User searches or scans a product during billing. <<includes>> Login.
UC-07	Generate Bill	Owner / Staff	User adds items and completes a sale. <<includes>> Search Product.
UC-08	Record Credit Sale	Owner / Staff	Bill is recorded against a customer's udhar instead of immediate payment. <<extends>> Generate Bill.
UC-09	Add Customer	Owner	Owner creates a customer profile with an optional credit limit.
UC-10	Record Credit Repayment	Owner	Owner records a payment received against a customer's outstanding udhar.
UC-11	View Credit Ledger	Owner	Owner views a customer's full credit/repayment history.
UC-12	View Dashboard	Owner	Owner views sales, profit, alerts, and AI suggestions summary.
UC-13	View AI Suggestions	Owner	Owner views reorder, expiry-risk, and best-seller suggestions. <<includes>> View Dashboard.

UC-ID	Use Case Name	Primary Actor	Brief Description
UC-14	Generate Sales Report	Owner	Owner downloads a sales/profit report for a chosen date range.

3.3 Key Relationships

Relationship	Type	Explanation
Search Product → Login	<<includes>>	A user MUST be logged in to search/bill a product. Login is always invoked.
Generate Bill → Search Product	<<includes>>	Every bill requires at least one product to be searched/scanned and added.
Record Credit Sale → Generate Bill	<<extends>>	Credit recording only happens when the payment mode chosen on a bill is 'Credit'.
View AI Suggestions → View Dashboard	<<includes>>	AI suggestions are always shown as part of the dashboard view.
Owner → Staff/Cashier	Generalisation	Owner inherits all Staff use cases (billing) and adds administrative capabilities.

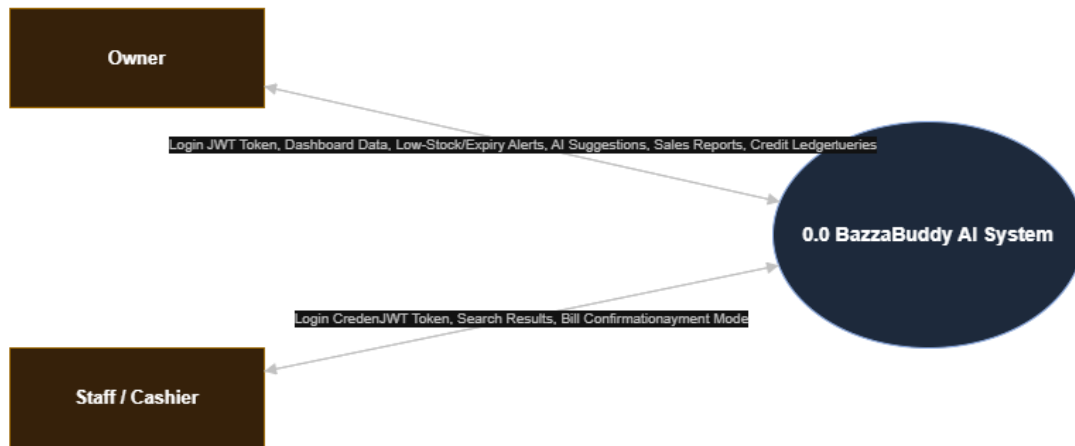


4. Data Flow Diagrams (DFD)

4.1 DFD Level 0 – Context Diagram

Level 0 shows the entire BazaarBuddy AI system as a single process ('black box') with all external entities and the data that flows in and out. There is exactly ONE process circle.

External Entity	Data Flow Direction	Data Flows
Owner	→ System (IN)	Login Credentials, Product Data, Batch/Purchase Entry, Customer Data, Credit Repayment Entry, Report Queries
Owner	← System (OUT)	JWT Token, Dashboard Data, Low-Stock/Expiry Alerts, AI Suggestions, Sales Reports, Credit Ledger
Staff / Cashier	→ System (IN)	Login Credentials, Product Search Query, Sale/Bill Entry, Payment Mode
Staff / Cashier	← System (OUT)	JWT Token, Search Results, Bill Confirmation



Note: DFD Level 0 shows the entire system as ONE process (black box) with all external entities and their data flows.

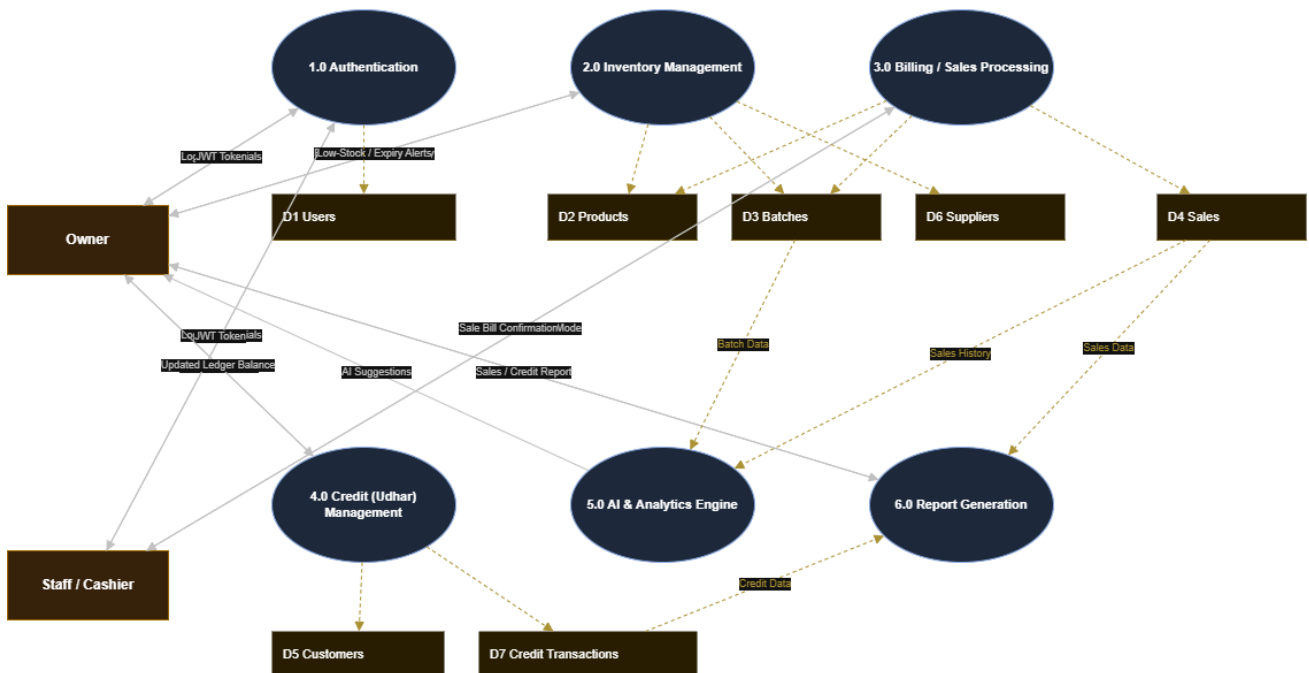
4.2 DFD Level 1 – Process Decomposition

Level 1 expands the single BazaarBuddy AI process into 6 sub-processes. Every external entity and data flow from Level 0 must appear at the boundary of Level 1 (balancing rule).

Process	Name	Inputs	Outputs
1.0	Authentication	Login Credentials	JWT Token, Role → D1 Users
2.0	Inventory Management	Product Data, Batch/Purchase Entry	Stock Updates, Low-Stock/Expiry Alerts → D2 Products, D3 Batches, D6 Suppliers
3.0	Billing / Sales Processing	Sale Items, Payment Mode	Bill Confirmation, Stock Deduction → D4 Sales, D2 Products, D3 Batches
4.0	Credit (Udhar) Management	Credit Sale Flag, Repayment Entry	Updated Ledger Balance → D5 Customers, D7 Credit Transactions
5.0	AI & Analytics Engine	Sales History from D4, Batches from D3	Reorder Suggestions, Expiry-Risk Flags, Best-Seller List → Owner
6.0	Report Generation	Queries from Owner, D4 Sales, D7 Credit Transactions	Sales/Profit Report, Credit Report → Owner

4.3 Data Stores

ID	Data Store Name	Contents
D1	Users	userID, name, email, hashedPassword, role (Owner/Staff), phone, isActive
D2	Products	productID, name, barcode, categoryID, unit, costPrice, sellingPrice, reorderLevel
D3	Batches	batchID, productID(FK), supplierID(FK), quantity, purchasePrice, expiryDate, receivedDate
D4	Sales	saleID, userID(FK), customerID(FK, optional), totalAmount, paymentMode, createdAt
D5	Customers	customerID, name, phone, creditLimit, outstandingBalance
D6	Suppliers	supplierID, name, contact, address
D7	CreditTransactions	transactionID, customerID(FK), saleID(FK, optional), amount, type (credit/payment), date



5. Entity-Relationship (ER) Diagram

5.1 Entities and Attributes

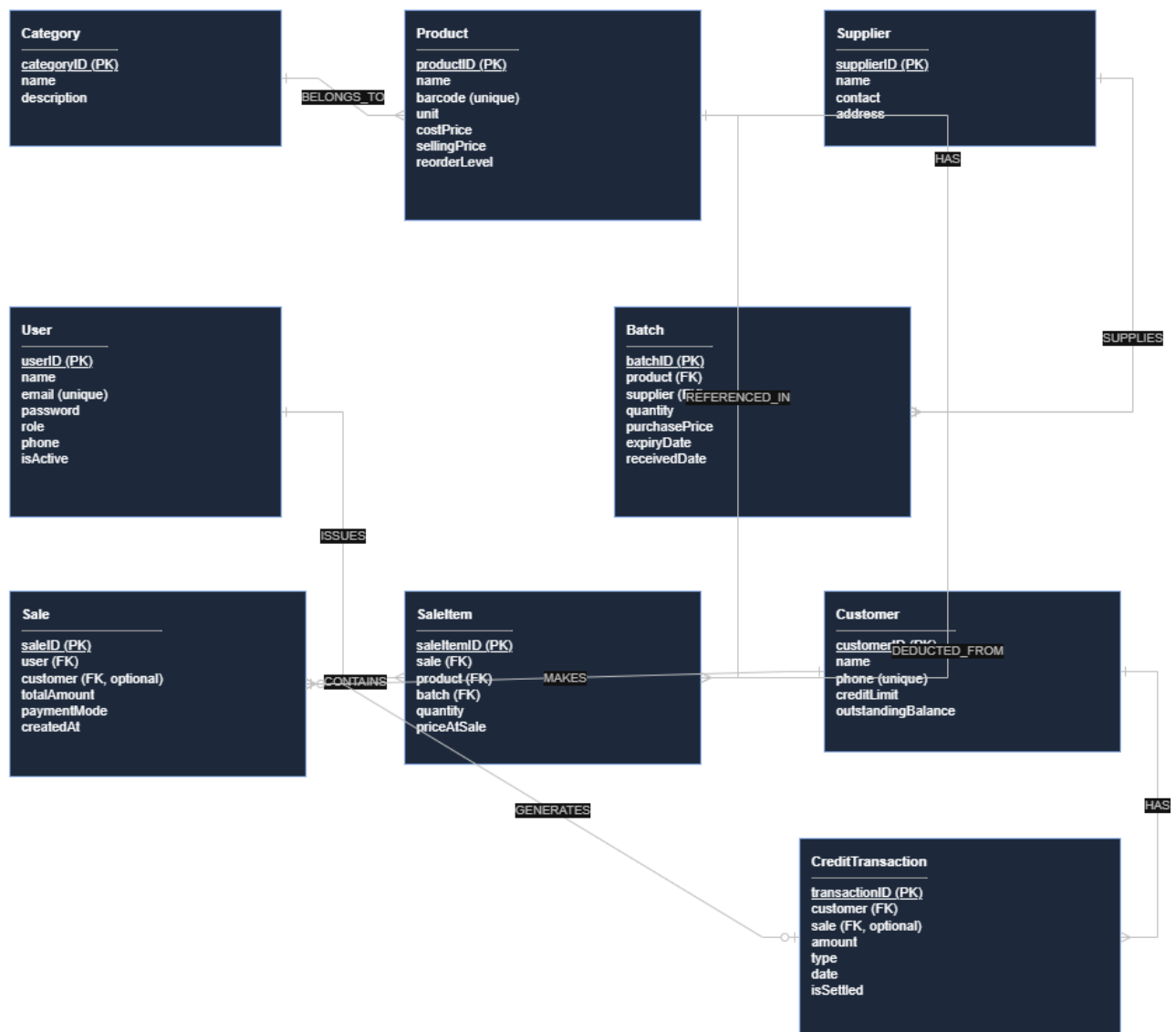
Entity	Attributes	Primary Key	Notes
User	userID, name, email, password, role, phone, isActive	userID	role: Owner Staff
Product	productID, name, barcode, unit, costPrice, sellingPrice, reorderLevel	productID	barcode must be UNIQUE across all products
Category	categoryID, name, description	categoryID	E.g., Grocery, Dairy, Snacks, Household
Supplier	supplierID, name, contact, address	supplierID	A supplier can supply many batches (1:M)
Batch	batchID, quantity, purchasePrice, expiryDate, receivedDate	batchID	One product can have many batches, each with its own expiry (1:M)
Sale	saleID, totalAmount, paymentMode, createdAt	saleID	Header record for a bill; issued by a User, optionally against a Customer
SaleItem	saleItemID, quantity, priceAtSale	saleItemID	Junction table for Sale CONTAINS Product (M:N resolved)
Customer	customerID, name, phone, creditLimit, outstandingBalance	customerID	Only needed for credit (udhar) or regular customers
CreditTransaction	transactionID, amount, type, date, isSettled	transactionID	type: credit (udhar given) payment (repayment received)

5.2 Relationships and Cardinality

Entity A	Relationship	Entity B	Resolution / Foreign Keys
User	ISSUES (1:M)	Sale	userID FK in Sale table – which Owner/Staff created the bill
Sale	CONTAINS (M:N)	Product	Resolved to SaleItem table: (saleID FK, productID FK, batchID FK, quantity, priceAtSale)
Product	HAS (1:M)	Batch	productID FK in Batch table. A product can have multiple batches with different expiry dates.
Supplier	SUPPLIES (1:M)	Batch	supplierID FK in Batch table.

Entity A	Relationship	Entity B	Resolution / Foreign Keys
Product	BELONGS_TO (M:1)	Category	categoryID FK added to Product table.
Customer	MAKES (1:M, optional)	Sale	customerID FK in Sale table, nullable for cash sales with no linked customer.
Customer	HAS (1:M)	CreditTransaction	customerID FK in CreditTransaction table.
Sale	GENERATES (1:1, optional)	CreditTransaction	saleID FK in CreditTransaction table. Only created when paymentMode = 'Credit'.

FIGURE 10-10 Entity-Relationship Model



Legend: —| = one —O< = zero-or-many —O| = zero-or-one (crow's foot notation)

6. Database Schema

BazzaBuddy AI uses MongoDB with Mongoose ODM. Below are the Mongoose schemas for the primary collections. The schemas implement the ER diagram from Section 5.

6.1 User Schema

Field	Type	Constraints	Notes
_id	ObjectId	PK (auto)	MongoDB auto-generated
name	String	required	Full name of the user
email	String	required, unique	Used for login; lowercase, trimmed
password	String	required, minLen:8	bcrypt hashed — never stored plain
role	String (enum)	required	Values: Owner Staff
phone	String	optional	10-digit mobile number
isActive	Boolean	default: true	Set false to deactivate a staff account

6.2 Product Schema

Field	Type	Constraints	Notes
_id	ObjectId	PK (auto)	MongoDB auto-generated
name	String	required, indexed	Text index for search
barcode	String	unique, sparse	Optional if product has no printed barcode
category	ObjectId	ref: Category	FK to Category collection
unit	String	required	E.g., kg, piece, litre, packet
costPrice	Number	required, min: 0	Price paid to supplier (latest)
sellingPrice	Number	required, min: 0	Price charged to customer
reorderLevel	Number	required, min: 0	Triggers low-stock alert when total quantity falls below this

6.3 Batch Schema

Field	Type	Constraints	Notes
_id	ObjectId	PK (auto)	MongoDB auto-generated
product	ObjectId	required, ref: Product	FK — which product this batch belongs to
supplier	ObjectId	ref: Supplier	FK — which supplier the batch was purchased from
quantity	Number	required, min: 0	Remaining quantity in this batch
purchasePrice	Number	required, min: 0	Cost price for this specific batch
expiryDate	Date	required	Used for expiry alerts and FIFO deduction
receivedDate	Date	default: now()	Date stock was received

6.4 Sale & SaleItem Schema

Field	Type	Constraints	Notes
_id	ObjectId	PK (auto)	MongoDB auto-generated (Sale)
user	ObjectId	required, ref: User	FK – Owner/Staff who created the bill
customer	ObjectId	optional, ref: Customer	FK – null for walk-in cash sales
items	[SaleItem]	required, min length: 1	Embedded array: {product FK, batch FK, quantity, priceAtSale}
totalAmount	Number	required, min: 0	Sum of all item subtotals
paymentMode	String (enum)	required	Values: Cash UPI Card Credit
createdAt	Date	default: now()	Date/time of sale

6.5 Customer & CreditTransaction Schema

Field	Type	Constraints	Notes
_id	ObjectId	PK (auto)	MongoDB auto-generated (Customer)
name	String	required	Customer's full name
phone	String	required, unique	Used to look up the customer at billing
creditLimit	Number	default: 0	Maximum udhar allowed; 0 means no credit allowed
outstandingBalance	Number	default: 0	Running total owed by this customer
transaction.type	String (enum)	required	Values: credit payment (CreditTransaction collection)
transaction.amount	Number	required, min: 0	Amount of the credit given or payment received

6.6 Supplier & Category Schemas

Collection	Fields	Notes
Supplier	name, contact, address	Referenced from Batch.supplier; tracks who stock was purchased from.
Category	name (unique), description	Referenced from Product.category; e.g., Grocery, Dairy, Snacks, Household.

6.7 Normalisation Notes

- 1NF: All attributes are atomic. No multi-valued fields (a Sale's line items are stored as an embedded array of sub-documents, not comma-separated text).
- 2NF: No partial dependencies. All non-key attributes depend on the full primary key of their collection.
- 3NF: No transitive dependencies. Supplier and batch-level purchase price are kept separate from Product's current sellingPrice.

- M:N resolved: Sale-Product M:N resolved via the embedded SaleItem array (with product and batch references). Customer credit history resolved via the separate CreditTransaction collection rather than embedding it in Customer.

7. System Architecture Diagram

BazzaBuddy AI follows a 3-tier architecture: Presentation (React.js) → Application (Node.js + Express.js) → Data (MongoDB Atlas), with an AI/Analytics layer sitting alongside the Application tier. JWT authentication with role checks (Owner vs Staff) is applied as a cross-cutting concern at the API layer.

7.1 Architecture Components

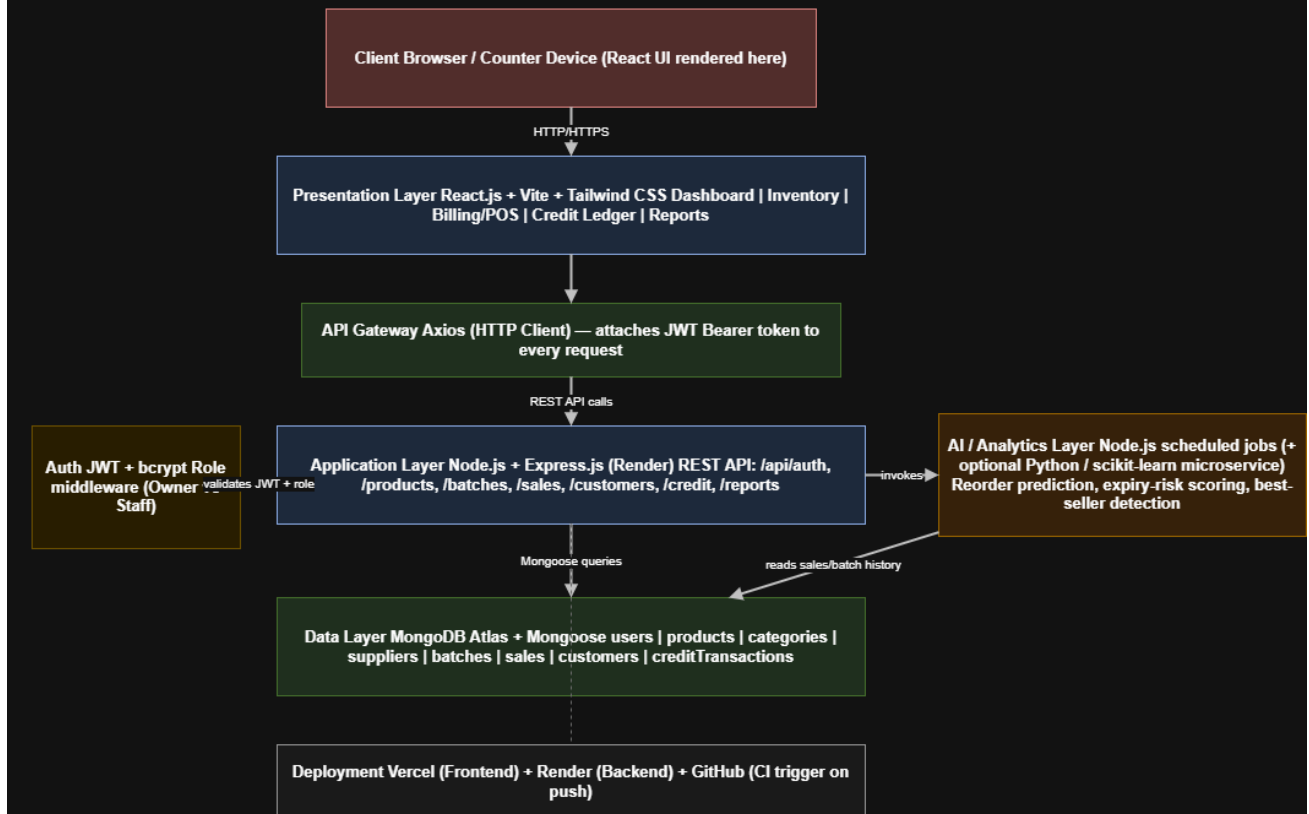
Layer	Component	Responsibility
Client	Browser / Counter Device	Renders React UI at the billing counter or back office. Sends HTTP requests via Axios. Stores JWT in memory / httpOnly cookie.
Presentation	React.js + Vite (Vercel)	Dashboard, Inventory, Billing/POS, Credit Ledger, Reports components. React Router v6 for SPA routing. Tailwind CSS for styling.
API Gateway	Axios (HTTP Client)	Attaches JWT Bearer token to every protected request. Base URL points to Express backend.
Application	Node.js + Express.js (Render)	REST API: /api/auth, /api/products, /api/batches, /api/sales, /api/customers, /api/credit, /api/reports. JWT + role middleware guards protected routes.
AI / Analytics Layer	Node.js scheduled jobs (+ optional Python/scikit-learn microservice)	Runs daily to compute reorder suggestions, expiry-risk scores, and best-seller/dead-stock lists from sales history.
Data	MongoDB Atlas + Mongoose	Collections: users, products, categories, suppliers, batches, sales, customers, creditTransactions. Mongoose schemas enforce validation.
Auth	JWT + bcrypt	JWT issued at login (24h expiry, role embedded). Middleware on every protected route validates token and role. bcrypt hashes passwords before storage.
Deployment	Vercel + Render + GitHub	GitHub push triggers auto-deploy. Vercel hosts React (vercel.json rewrite for React Router). Render hosts Express (env vars in dashboard).

7.2 API Routes

Method	Route	Auth	Description
POST	/api/auth/register	Owner (bootstrap)	Register the store owner account
POST	/api/auth/login	Public	Login; returns role-tagged JWT token
POST	/api/auth/staff	Owner JWT	Create a Staff/Cashier account
GET	/api/products	Owner/Staff JWT	List/search products; supports ?search= and ?barcode= query params
POST	/api/products	Owner JWT	Add a new product to catalogue

Method	Route	Auth	Description
PUT	/api/products/:id	Owner JWT	Update product details
DELETE	/api/products/:id	Owner JWT	Delete a product (only if no active batches)
POST	/api/batches	Owner JWT	Record a new stock batch (purchase entry) with expiry date
GET	/api/batches/expiring	Owner JWT	List batches expiring within the next 7 days
POST	/api/sales	Owner/Staff JWT	Create a new bill; deducts stock and updates credit ledger if applicable
GET	/api/sales/today	Owner/Staff JWT	Get today's sales list and total
POST	/api/customers	Owner JWT	Add a new customer with optional credit limit
GET	/api/customers/:id/ledger	Owner JWT	Get a customer's full credit/repayment ledger
POST	/api/credit/repay	Owner JWT	Record a repayment against a customer's outstanding credit
GET	/api/analytics/dashboard	Owner JWT	Get today's sales, profit, alerts, and best-sellers for the dashboard
GET	/api/analytics/ai-suggestions	Owner JWT	Get AI-generated reorder, expiry-risk, and best-seller suggestions
GET	/api/reports/sales	Owner JWT	Generate a sales/profit report for a given date range

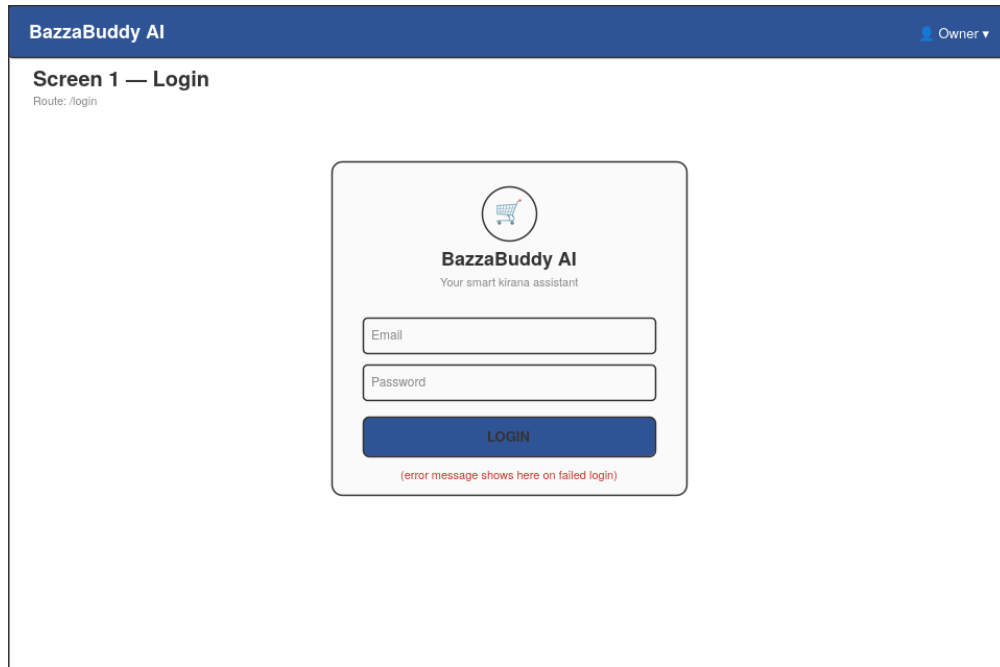
System Architecture Diagram — BazzaBuddy AI



8. Wireframes

Six screen wireframes are required. Each wireframe must be drawn in Figma or on A4 paper and uploaded to /docs/wireframes/ on GitHub. Below is the specification for each screen.

8.1 Screen 1 – Login (Route: /login)

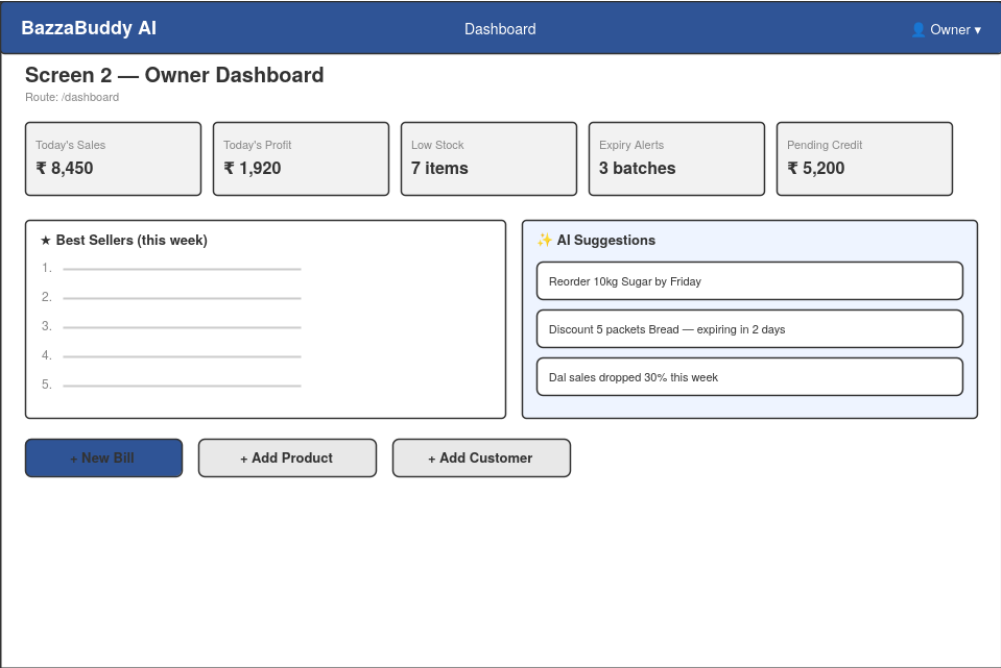


The wireframe for the login screen is titled "Screen 1 — Login" with the route "/login". It features a dark blue header bar with the text "BazzaBuddy AI" on the left and a user role indicator "Owner" with a dropdown arrow on the right. The main content area is white and contains a centered login form. The form has a light gray background and rounded corners. At the top of the form is a circular logo with a shopping cart icon. Below the logo is the text "BazzaBuddy AI" and "Your smart kirana assistant". The form includes two input fields: "Email" and "Password". Below these fields is a blue "LOGIN" button. At the bottom of the form, a red error message is displayed: "(error message shows here on failed login)".

Low-fidelity reference wireframe – redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Logo & Tagline	BazzaBuddy AI logo (centre). Tagline: 'Your smart kirana assistant'.
Login Form	Email input + Password input + Login button. Error message shown below form on failure.
Role Badge	After login, top bar shows the logged-in user's name and role (Owner / Staff).

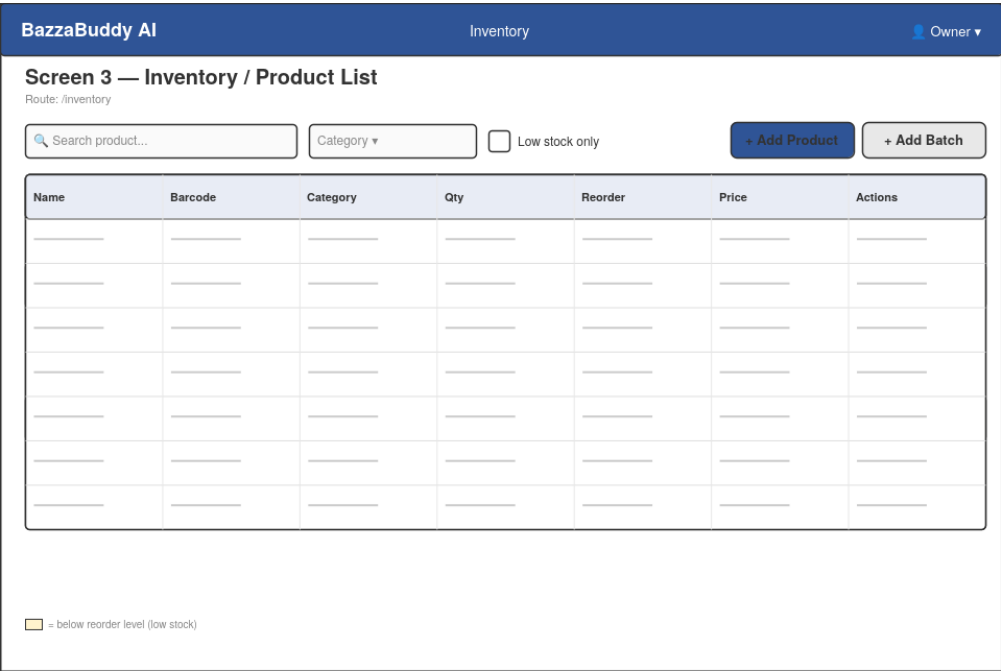
8.2 Screen 2 – Owner Dashboard (Route: /dashboard)



Low-fidelity reference wireframe — redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Summary Cards	Cards: Today's Sales (Rs.) Today's Profit (Rs.) Low Stock Count Expiry Alerts Pending Credit (Rs.).
Best Sellers Widget	Small ranked list of top 5 products by quantity sold this week.
AI Suggestions Panel	Card list: 'Reorder 10kg Sugar by Friday', 'Discount 5 packets of Bread — expiring in 2 days', etc.
Quick Actions	Buttons: New Bill Add Product Add Customer.

8.3 Screen 3 — Inventory / Product List (Route: /inventory)



Low-fidelity reference wireframe — redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Filter Bar	Search input + Category dropdown + 'Low stock only' toggle.
Product Table	Columns: Name Barcode Category Quantity Reorder Level Selling Price Actions (Edit/Delete). Low-stock rows highlighted amber.
Add Product Button	Opens a modal form: name, barcode, category, unit, cost/selling price, reorder level.
Batch Entry Button	Opens a modal to record a new batch: supplier, quantity, purchase price, expiry date.

8.4 Screen 4 – Billing / POS (Route: /billing)

BazzaBuddy AI

Billing

Owner ▼

Screen 4 — Billing / POS

Route: /billing

Product	Qty	Price	Subtotal	Remove
TOTAL			₹ 1,240	

Payment Mode

Cash

UPI

Card

Credit

Customer (if Credit):

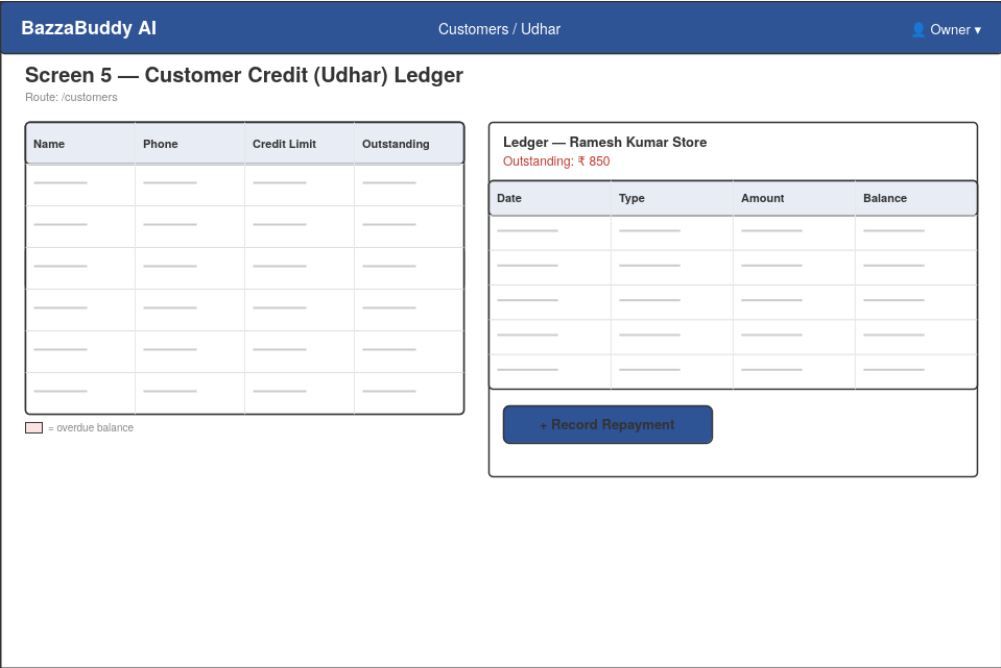
Search customer...

COMPLETE BILL

Low-fidelity reference wireframe – redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Product Search	Search bar with barcode-scan icon. Typing/scanning adds matching product to the cart.
Cart Table	Columns: Product Qty (editable) Price Subtotal Remove. Live total at the bottom.
Payment Panel	Payment mode buttons: Cash UPI Card Credit. If Credit selected, customer search/select field appears.
Complete Bill Button	Finalises the sale, deducts stock, updates credit ledger if applicable, shows printable bill.

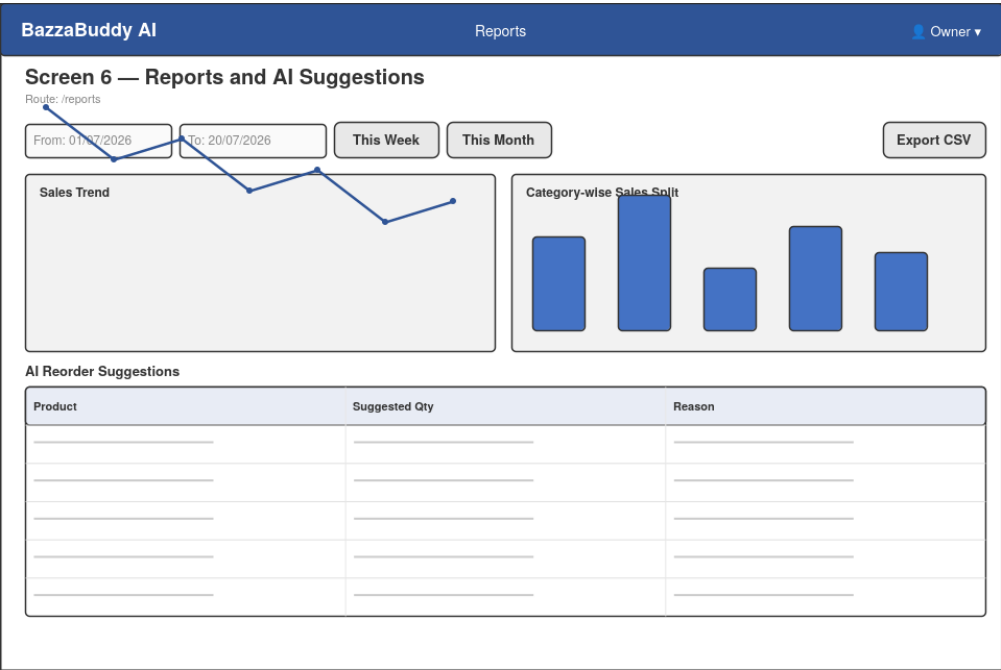
8.5 Screen 5 – Customer Credit (Udhar) Ledger (Route: /customers)



Low-fidelity reference wireframe — redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Customer List	Table: Name Phone Credit Limit Outstanding Balance. Overdue balances highlighted red.
Ledger View	Selecting a customer opens their full history: date type (credit/payment) amount running balance.
Record Repayment	Button opens a small form to record a payment received, reducing outstanding balance.

8.6 Screen 6 — Reports & AI Suggestions (Route: /reports)



Low-fidelity reference wireframe — redraw/refine in draw.io or Figma using this as a guide.

Element	Description
Date Range Filter	From/To date pickers + 'This Week' / 'This Month' quick buttons.
Sales Trend Chart	Line chart of daily sales/profit over the selected range.
Category Split	Pie/bar chart of sales by category.
AI Suggestions Table	Table: Product Suggested Reorder Qty Reason (e.g., 'sells out every 5 days'). Export as CSV button.

9. Project Gantt Chart

The Gantt chart tracks all 5 project phases across 50 sessions. Each row is a phase; each column block represents a group of sessions.

Phase	S01–S05	S06–S10	S11–S15	S16–S20	S21–S25	S26–S30	S31–S35	S36–S40	S41–S45	S46–S50
P1 – Design Unit (CO1)	■ ■ ■ ■ ■	■ ■ ■ ■ ■								
P2 – React.js Frontend (CO2)			■ ■ ■ ■ ■	■ ■ ■ ■ ■						
P3 – Node.js Backend (CO3)					■ ■ ■ ■ ■	■ ■ ■ ■ ■				
P4 – Full-Stack + AI Integration (CO4)							■ ■ ■ ■ ■	■ ■ ■ ■ ■		
P5 – Testing & Deployment (CO5)									■ ■ ■ ■ ■	■ ■ ■ ■ ■

9.1 Milestones

ID	Milestone	Due Session	Deliverable	Marks / Evaluation
M1	Design Document	S10	SRS + Use Case + DFD + ER + Schema + Architecture + Wireframes + Gantt (PDF + GitHub tag v1.0-design-doc)	10 marks (IPE)
M2	React Frontend Sprint	S20	Dashboard, Inventory, Billing, Credit, Reports screens functional, routed, pushed to feature/frontend branch.	GitHub Activity (commits)
M3	Node.js Backend Sprint	S30	All REST API endpoints working, JWT + role auth live, tested in Postman, pushed to feature/backend.	GitHub Activity (commits)
M4	Integrated App + AI Demo	S40	Full-stack app with billing, inventory, credit, and AI suggestions integrated; live demo to faculty. Merged to dev branch.	15 marks (MSPE)
M5	Final Submission (FPE)	S50	Deployed app (Vercel + Render), Project Report (min 20 pages), User Manual, Presentation Deck, Individual Viva.	50 marks (FPE)

10. M1 Design Document Submission Checklist

Complete this checklist before submitting the Design Document at Session S10.

✓	Item	Location	Marks (M1 Rubric)
[]	SRS: 15+ Functional Requirements + 5+ Non-Functional Requirements + Scope + Constraints	Section 2 of doc	20%
[]	Use Case Diagram: 3 actors (Owner, Staff/Cashier, System), 14 use cases, include/extend/generalisation	/docs/diagrams/use-case.png	15%
[]	DFD Level 0: Exactly 1 process circle, 2 external entities, all data flows labelled	/docs/diagrams/dfd-l0.png	10%
[]	DFD Level 1: 6 sub-processes (1.0–6.0), 7 data stores (D1–D7), balanced with Level 0	/docs/diagrams/dfd-l1.png	10%
[]	ER Diagram: 9 entities, primary keys underlined, all M:N relationships resolved to junction tables	/docs/diagrams/er-diagram.png	20%
[]	Database Schema: Mongoose schemas for all core collections (User, Product, Batch, Sale, Customer, CreditTransaction)	Section 6 of doc	10%
[]	System Architecture Diagram: Browser → React → Axios → Express → Mongoose → MongoDB + JWT + AI layer	/docs/diagrams/architecture.png	5%
[]	Wireframes: All 6 screens with route paths labelled (Login, Dashboard, Inventory, Billing, Customers, Reports)	/docs/wireframes/	10%
[]	Gantt Chart: 5 phases × 10 sessions, 5 milestones marked with session numbers and deliverables	Section 9 of doc	0% (Required)
[]	GitHub repository created with branching strategy: main → dev → feature/frontend, feature/backend	GitHub README	(Required)
[]	GitHub tagged release created: v1.0-design-doc	GitHub Releases	(Required)
[]	Document has group number and all member names on cover page	Cover Page	(Required)
[]	Document is minimum 15 pages in length	Full document	(Required)

GitHub Repository URL: <https://github.com/riddhi1120/BazaarBuddy-AI.git>

GitHub Tag: v1.0-design-doc | Submission Date: _____

Member 1 Signature:	Member 2 Signature:	Member 3 Signature:
_____	_____	_____